AIGC 5500 | MODULE 04 | 4% OF FINAL GRADE

Lab 04: What Recurrence Can and Cannot Remember

Due date posted on Blackboard

Module 04 made four claims about recurrent networks: they are trainable but fragile, the gradient through time vanishes or explodes geometrically, gates give the architecture a long-range path, and initialization decides whether training starts out using that path. In this lab you test those claims yourself, on a delayed-recall task where the right answer is known. You will measure how gradients behave through time, compare recurrent cells under controlled conditions, check which fixes actually matter, and finish by measuring the two limits that recurrence cannot escape. What is being assessed is whether you can tell a claim from a measurement, and whether your comparisons are controlled well enough to support your conclusions.

Starter code

You are given lab04_starter.py, containing the delayed-recall task, a recurrent model whose forget gate or update gate can start open, and a training loop with gradient clipping and a fixed validation set. Everything needed to train and evaluate is provided. You write the measurements, the comparisons, the timing and the conclusion. Do not edit the starter file directly; copy it into your repository and import from it, or copy what you need into your own work.

The task: step 0 of each sequence carries a class label (4 classes) as a one-hot vector, every later step carries only noise, and the model must report the label at the last step. Chance accuracy is 0.25. The longer the sequence, the further the label has to be carried.

```
from lab04_starter import make_recall_batch, RecallNet, train_recall

# Train one configuration: cell type, sequence length, seed, starting gate bias.
accuracy, history = train_recall("lstm", seq_len=30, seed=0, gate_bias=3.0)
print(accuracy)    # validation accuracy; chance is 0.25

# gate_bias=None keeps PyTorch's default initialization.
# clip=None turns gradient clipping off (the default is clip=1.0).
accuracy, history = train_recall("rnn", seq_len=30, seed=0, clip=None)
```

Run lab04_starter.py once, unmodified, to confirm your environment works before changing anything. It trains a plain recurrent layer on a 10-step version of the task and should report an accuracy near 1.0.

What to build

Four pieces, in order.

1. **A benchmark table.** Train five configurations on sequence lengths 10, 30 and 60, using at least three seeds (0, 1 and 2) for each: a plain RNN, an LSTM at default initialization, an LSTM with forget-gate bias 3, a GRU at default initialization, and a GRU with update-gate bias 3. Keep hidden size, number of updates, batch size, learning rate and clipping at the starter's defaults. Report the validation accuracy for every seed, plus the mean, in one table.
2. **A gradient-through-time measurement.** Write your own function that measures the gradient norm reaching each input step when the loss reads only the last step of a 60-step sequence (hidden size 32, batch 16, 8 input features), taking the median over at least five seeds. Apply it to the plain RNN, the LSTM at default initialization, and the LSTM with forget-gate bias 3. Plot the three curves on a log scale, and report the values 0, 9, 29 and 59 steps back from the loss.
3. **Two controlled fixes.** First, a forget-bias sweep: an LSTM with forget-gate bias 0 (the default), 1, 2 and 3 at length 30, three seeds each. Second, a clipping comparison: the plain RNN at length 30 with gradient clipping on and off, three seeds each. Write down your prediction for the clipping comparison in your README before you run it. Then report what happened and explain it using your gradient measurement from piece 2.

4. **Limits and a conclusion.** Time a forward and backward pass of an LSTM at sequence lengths 50, 100, 200 and 400, taking the best of three repeats at each length. Then write a conclusion that says what gates changed, what initialization changed, what clipping changed, and which two limits of recurrence your measurements illustrate. For each limit, say how the direct connections previewed at the end of Module 04 would address it.

Requirements

Your submission must do all of the following. Each is checked directly during marking.

- Compare configurations under identical conditions: the same data generator, the same validation set, the same number of updates, the same optimizer, learning rate, batch size and clipping setting. The only things that may differ between two compared runs are the cell type and the starting gate bias (and, in the clipping comparison, whether clipping is on).
- Report every seed, not only the mean. At least three seeds per cell in the benchmark table, the forget-bias sweep and the clipping comparison.
- Set the gate bias on the correct slice: the forget gate of an LSTM and the update gate of a GRU are both slice [H : 2H] of each bias vector, and the effective bias is the sum of bias_ih and bias_hh. State in your README how you confirmed you changed the intended gate.
- Measure the gradient with the loss reading only the last step of the sequence, as a norm per input step, as a median over at least five seeds, plotted on a log scale.
- Write your clipping prediction in your README before running the comparison, and keep it in the final README even if it turned out to be wrong.
- Explain the benchmark results through your gradient measurement, not just by stating which configuration scored higher. Quote accuracy relative to chance (0.25).
- Report surprising or inconvenient results honestly, such as a default configuration that learns on one seed, or a fix that does not help. A null result explained correctly is worth more than a flattering plot.
- Report the timing table, and state the two limits of recurrence your measurements illustrate, with the direct-connection idea from Module 04 that addresses each.

Watch out

PyTorch stores gate parameters stacked in one block, and the order is a library convention. An LSTM is laid out as (input, forget, cell, output); a GRU as (reset, update, new). Slicing the wrong block silently changes a different gate, and the code still runs. Print the slice you modified, and confirm that the value you expect is there, before you trust an initialization change.

Why hold everything else fixed, and why report seeds

If the open-gate LSTM is trained with a different learning rate, or for more updates, than the default LSTM, a gap between them says nothing about gates. This is the same principle behind the staged comparisons in Lab 02 and Lab 03. Seeds matter for a different reason: at sequence lengths between the easy case and the hard case, the same configuration can solve the task on one seed and stay at chance on another, so a mean can hide two different outcomes. That is why this lab asks for every seed.

Repository requirements

The repository is as much a part of the submission as the code. It must contain the following, and it must not contain your virtual environment.

Item	Requirement
.gitignore	Present, committed before your first code commit, and excluding .venv, __pycache__, .env, and any large output files
README.md	Explains the task, each of the four pieces, your clipping prediction, your results tables, and how to reproduce your results
requirements.txt	Generated with pip freeze, including torch and matplotlib
Your code	Your gradient measurement, the benchmark, the forget-bias sweep, the clipping comparison and the timing, as scripts or a single notebook, together with lab04_starter.py
Figures	The gradient-through-time plot saved as an image in the repository and referenced from the README
Commit history	At least three commits, each a coherent change, with messages that describe what changed

README template

You may use this structure directly. Replace the content, not the shape.

```
# What Recurrence Can and Cannot Remember

Measuring gradients through time and comparing recurrent cells on a
delayed-recall task.

## Setup
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt

## Run
python benchmark.py    # piece 1: 5 configs x lengths 10, 30, 60
python gradients.py    # piece 2: gradient per step, saves plot
python fixes.py        # piece 3: bias sweep and clipping
python timing.py       # piece 4: LSTM time against length

## Prediction (written before running the clipping comparison)
What I expected clipping to do for the plain RNN at length 30, and why.

## Results
Benchmark: accuracy per seed and mean, every configuration and length.
Gradients: values 0, 9, 29 and 59 steps back, and the plot.
Fixes: the forget-bias sweep and the clipping comparison, every seed.
Timing: LSTM time at lengths 50, 100, 200 and 400.

## Conclusion
What gates, initialization and clipping each changed, explained through
the gradient measurement; the two limits of recurrence your measurements
illustrate; and how direct connections would address each.

## Known limitations
Anything you are aware of that does not work, or that you would improve
with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 04 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

Instructor GitHub account

`hossein.pourmodheji@humber.ca`

Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Before you submit, run this check

Clone your own repository into a different folder, create a fresh virtual environment, install from requirements.txt, and reproduce every table and the gradient plot following only your README. The whole set of runs takes a few minutes on a laptop CPU. If everything completes cleanly and agrees with what you reported, your submission is complete. If it does not, you have found the problem before your marker did.